

Performance Testing

Date	7 July 2026
Team ID	xxxxxx
Project Name	AI Powered Debt Relief & Financial Recovery Platform
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, FastAPI Swagger UI, Browser Developer Tools
Type of Testing	Load Testing, API Response Testing, Performance Testing
Target Module / API	User Authentication, Financial Dashboard, Loan Management, Settlement Prediction APIs
Test Environment	Local Development Environment (React + FastAPI + SQLite)
Test Date	07 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	User Registration API	20	60	Users register successfully without errors
2	User Login API	20	60	Secure login with JWT token generated

3	Loan Management API	15	60	Loan records added, updated, deleted, and retrieved successfully
4	Financial Dashboard & AI APIs	10	60	Dashboard loads correctly and AI responses are returned within acceptable response time



Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.2 Seconds	Pass	Fast API responses
2	Response Time (Max)	< 5 seconds	3.4 Seconds	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 50 Req/sec	65 Req/Sec	Pass	Stable throughput under normal load
4	Error Rate	< 1%	0.2%	Pass	Very low error rate
5	CPU Utilization	< 80%	48%	Pass	Efficient backend processing
6	Memory Utilization	< 80%	55%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings

- The application responded efficiently during authentication and loan management operations.
- Dashboard data loaded quickly with acceptable response times.
- Financial calculations and settlement prediction APIs executed successfully.
- Backend remained stable under expected user load.
- No significant performance degradation was observed during testing.

Bottlenecks Identified

- AI response time depends on external API latency.
- SQLite is suitable for development but may require migration to MySQL or PostgreSQL for large-scale deployment.

Optimization Steps Taken

- Optimized REST API endpoints.
- Reused database connections through SQLAlchemy.
- Improved frontend API calls using Axios.
- Added proper exception handling to reduce unnecessary failures.
- Implemented modular backend architecture for better maintainability.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

